

Лабораторная работа

Основы работы в QNX Neutrino

Методические указания

1 Цель работы

Получение навыков практической работы в операционной системе реального времени QNX Neutrino. Изучение основных команд консоли операционной системы QNX, привитие навыков работы с файлами, каталогами и дисками.

2 Конфигурирование

2.1 Регистрация пользователя в системе

Вход пользователя в систему (правильнее сказать, регистрацию пользователя в системе) можно выполнить двумя способами - из командной строки или через графический интерфейс. Способ, который система предложит, зависит от того, был ли установлен соответствующий флажок при настройке видеодрайвера при первом старте **QNX Neutrino** после инсталляции. Если флажок не был установлен (т. е. операционной системе запрещено автоматически запускать графическую оболочку **Photon**), то в каталоге `/etc/system/config` был создан пустой файл с именем **nophoton**. Во время загрузки системы стартовые сценарии просто-напросто проверяют, существует ли файл **nophoton**. Создавая и удаляя этот файл, можно указывать - вручную или автоматически должна запускаться графическая оболочка **Photon**.

При командно-строковой регистрации, если введены правильные имя пользователя и пароль, то стартует командный интерпретатор **shell**. Этот интерпретатор называют *входным* (**login shell**).

В нем можно запускать различные команды, в том числе можно запустить **Photon** командой **ph** (при этом вводить пароль не потребуется).

Графическая оболочка **Photon** определяет - требовать регистрацию или нет. Она проверяет значение переменной окружения **logname**. Если вы зарегистрировались посредством **login**, то переменная **logname** будет содержать ваше регистрационное имя. Если же выполнялся автоматический запуск **Photon**, то значение переменной **logname** еще не задано (рис. 1). Отсюда вытекает интересная возможность для встроенных систем с графическим интерфейсом - можно в стартовых сценариях принудительно задать для переменной **logname** значение и сразу запускать **Photon** от имени какого-либо пользователя.

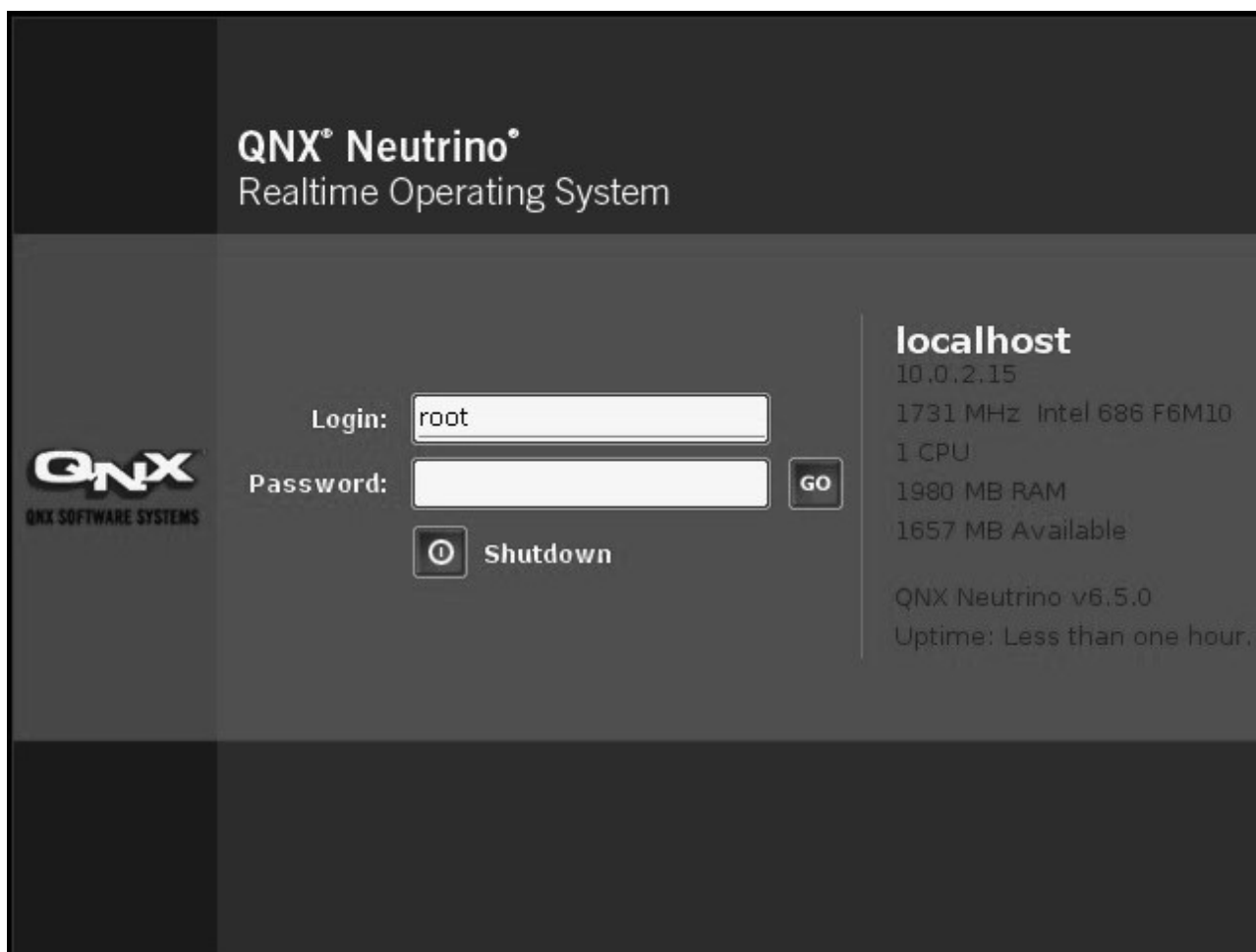


Рис. 1. Регистрация пользователя в системе

Для выхода из системы есть следующие способы:

- в графической среде - выбрать элемент **Log out** меню **Launch** (запустится утилита **phshutdown**);
- в **login shell** - выполнить команду **logout** или **exit**;
- выполнить команду **shutdown**.

Команду **shutdown** может использовать только суперпользователь (пользователь **root**). Она посылает сигнал **SIGPWR** ("сейчас будет отключено питание") всем выполняющимся процессам. Десять секунд ожидает их завершения и перезагружает систему. Если команде **shutdown** указать опцию **-f** (т. е. **fast** - "быстро"), то время ожидания будет сокращено до одной секунды. Возможно несколько вариантов поведения утилиты **shutdown**, задаваемых опцией **-s**:

- **system** - остановить систему для выключения питания;
- **reboot** - перезагрузить систему;
- **photon** - закрыть графическую оболочку **Photon** и продолжать работу в консоли;
- **user** - закончить пользовательский сеанс. При этом на экране появится регистрационное приглашение для входа в систему.

Если существует файл **/var/log/wtmp**, то утилита **shutdown** добавит в него запись о своей работе.

Утилита **phshutdown** по своей функциональности аналогична утилите **shutdown**. Она тоже посылает сигнал **SIGPWR** всем выполняющимся процессам, десять секунд ожидает их завершения и перезагружает систему, однако вызывать ее имеет право любой пользователь. Чтобы запретить использование этой утилиты всем пользователям, кроме **root**, необходимо создать пустой файл **/usr/photon/config/phshutdown.restrict**. Если существует файл **/var/log/wtmp**, то утилита **phshutdown** добавит в него запись о своей работе.

phshutdown предлагает выбрать один из трех вариантов действий:

- завершить сеанс **Photon**;

• остановить систему. В **QNX** не реализовано автоматическое выключение блока питания. Поэтому после появления на экране сообщения о том, что ОС **QNX** завершила работу, следует нажать кнопку **Power**;

- перезагрузить систему.

После запуска графической оболочки, на экране появляется так называемое рабочее пространство (**workspace**) **Photon**. Как работать в графической оболочке - интуитивно понятно. Внизу расположена панель задач, в левой ее части находится кнопка **Launch**. Нажатие кнопки **Launch** открывает меню, позволяющее вызывать различные прикладные программы. В правой части экрана находится меню быстрого запуска приложений - **Shelf** ("полка", рис. 2).

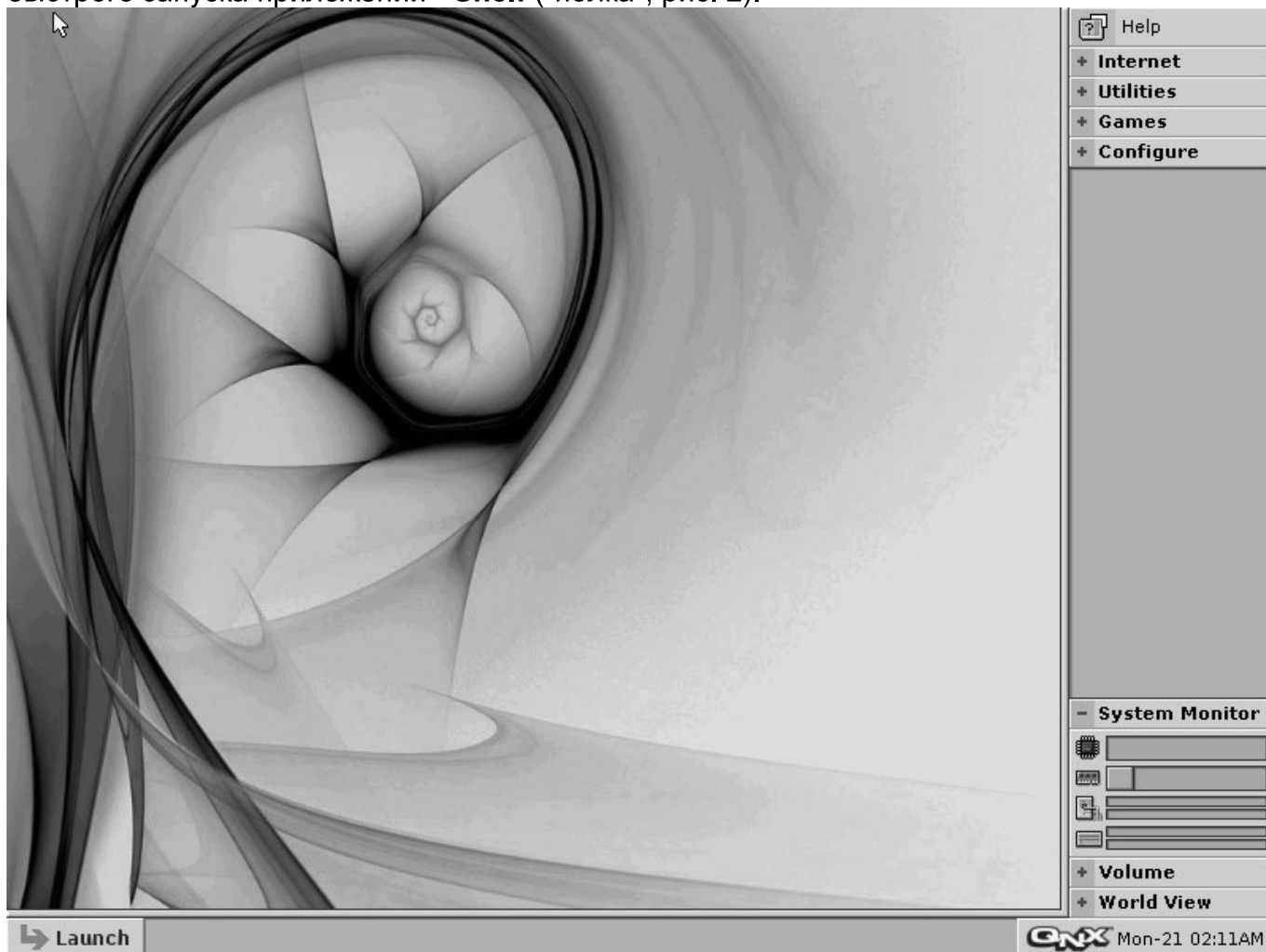


Рис. 2. Меню Shelf быстрого запуска приложений (справа)

Следует обратить внимание на следующие "часто используемые" программы:

- **Photon Terminal (pterm)** - штатный псевдотерминал (рис. 3), позволяющий работать с инструментами командной строки;

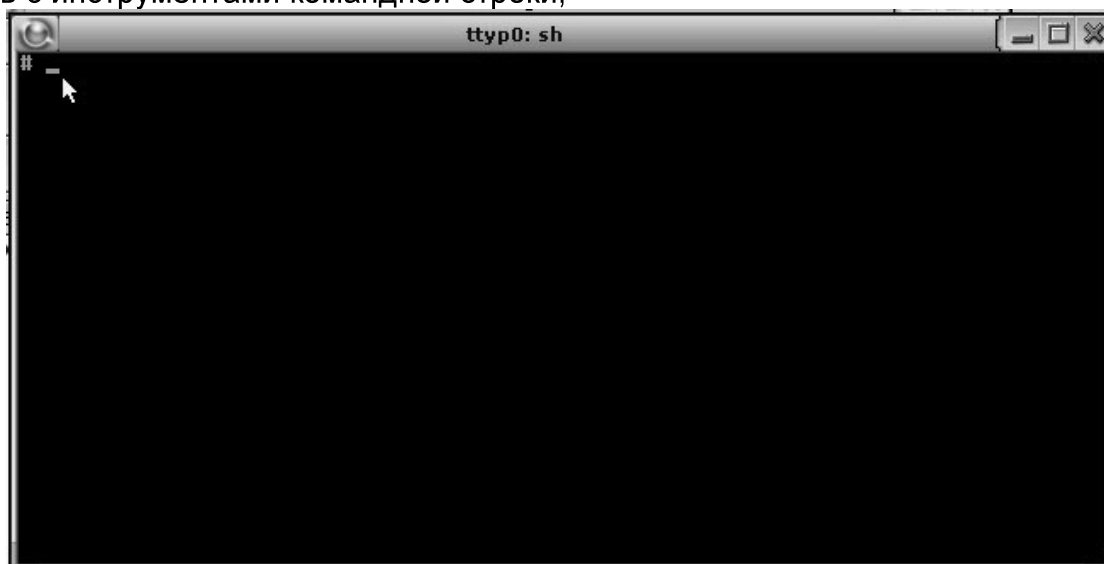


Рис. 3. Photon Terminal

- **Helpviewer** - программа доступа к штатной электронной документации (рис. 4). В составе QNX поставляется достаточно обширная и подробная документация;

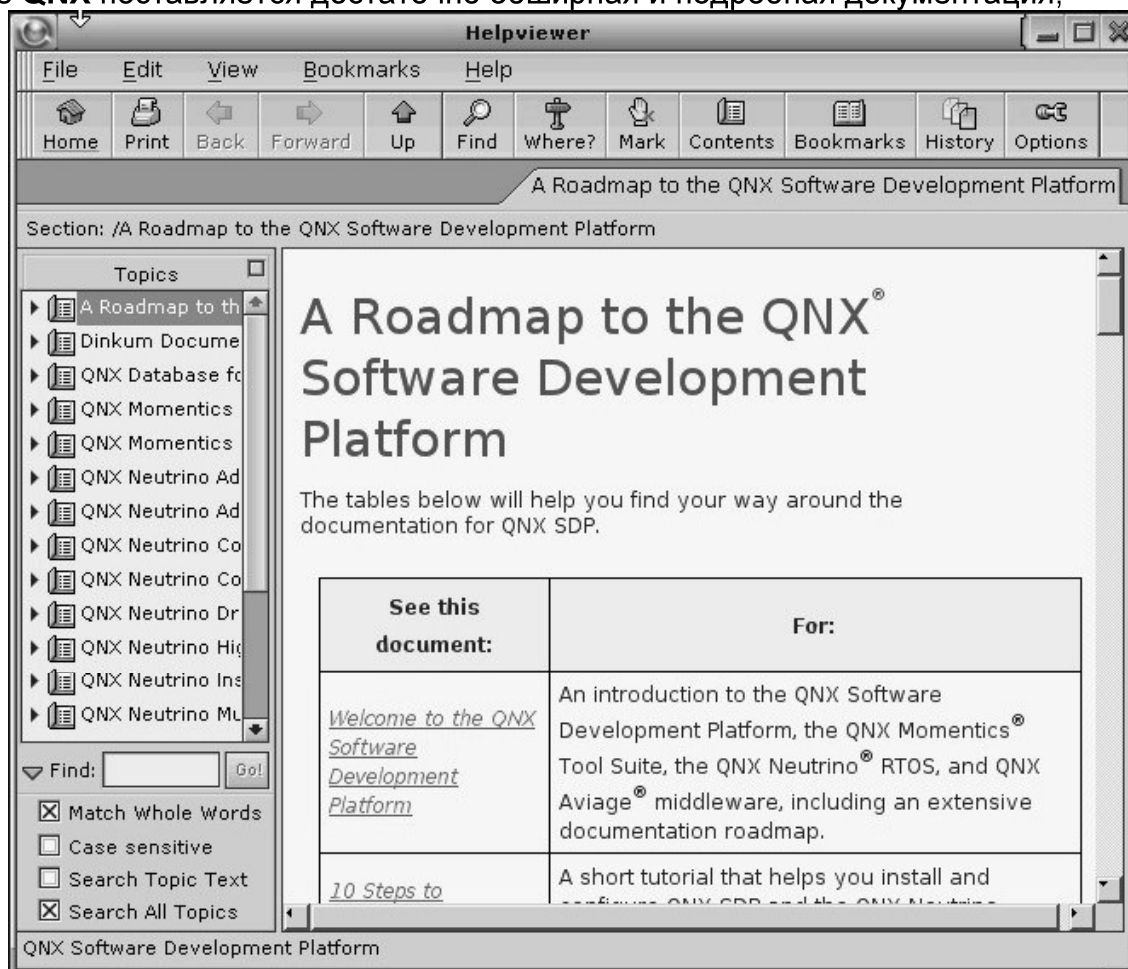


Рис. 4. Helpviewer

- **Photon File Manager (pfm)** - штатный файловый менеджер (рис. 5).

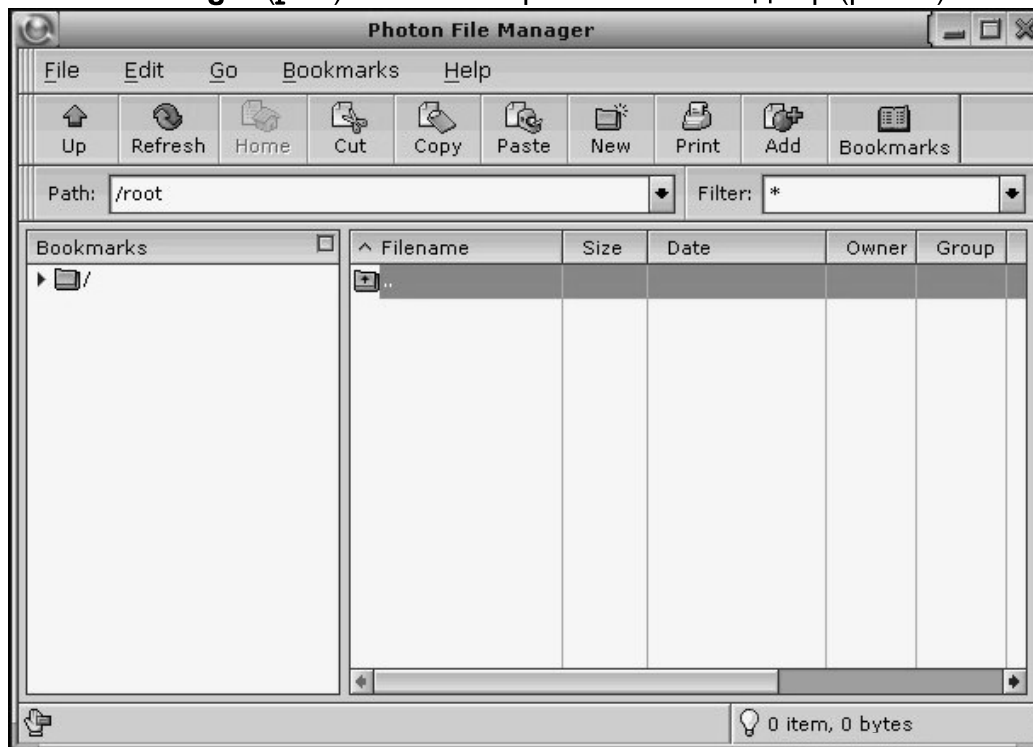


Рис. 5. Photon File Manager

2.2 Конфигурирование сети

Для нормальной работы следует выполнить несколько простых операций. Если ЭВМ подключена к сети, то потребуется знать **IP**-адрес и сетевую маску нашей машины, **IP**-адреса шлюза и сервера имен. Если вы - системный администратор, то эти сведения вам известны, в противном случае обратитесь к системному администратору. Однако, задать хотя бы имя хоста и его **IP**-адрес желательно даже для одиночного компьютера, потому что многие программы изначально рассчитаны для работы в сети и работают корректно только при наличии сетевых настроек.

Запустим программу конфигурирования сети, нажав кнопку **Network** в разделе **Configure** панели в правой части окна (меню **Shelf**) и вручную (**Manual**) зададим **IP**-адрес и маску подсети нашей машины в открывшемся окне **TCP/IP Configuration** (рис. 6).

При желании можно указать, чтобы ОС **QNX Neutrino** запрашивала параметры для своих сетевых интерфейсов у определенного **DHCP**¹-сервера. Разумеется, **DHCP**-сервер должен быть доступен и правильно сконфигурирован. **DHCP**-сервер может работать и на **QNX Neutrino** (см. описание программы **dhcpcd** в **Utilities Reference**).

¹ DHCP (Dynamic Host Configuration Protocol) - протокол динамической конфигурации хостов.

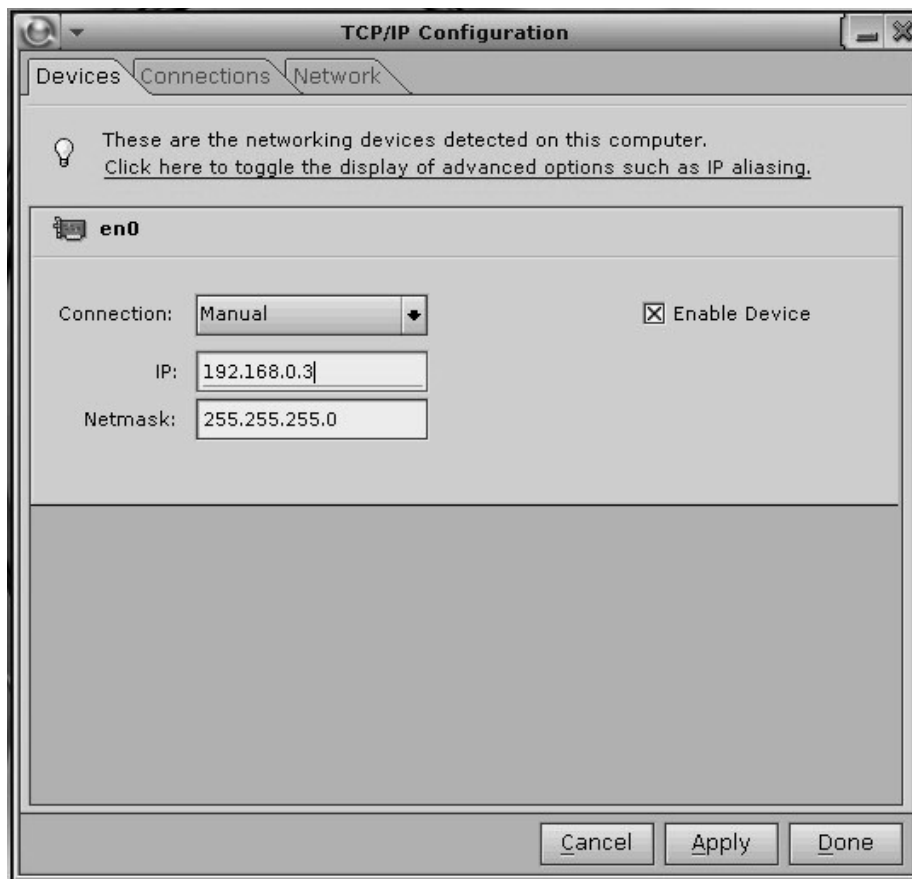


Рис. 6. Привязка IP-адресов к сетевым интерфейсам

Теперь перейдем на вкладку **Network** и введем имена ЭВМ (**Host Name**) и домена (**Domain Name**), IP-адреса шлюза (**Default Gateway**) и серверов имен (**Name Servers**) (рис. 7).

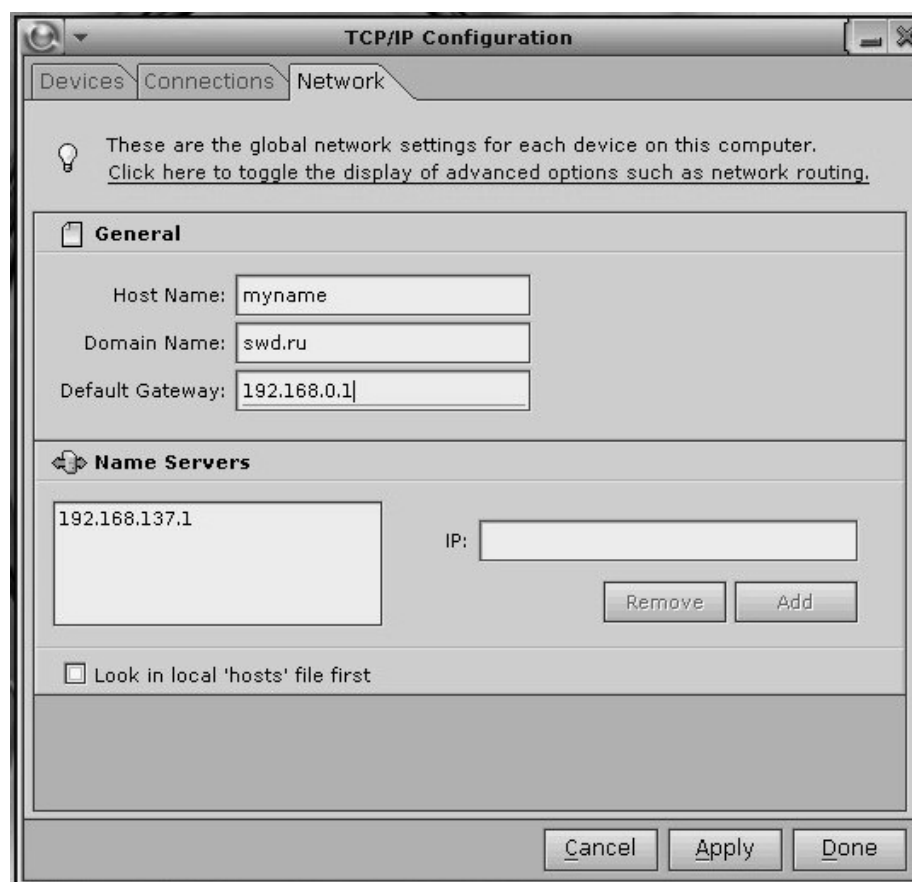


Рис. 7. Ввод имен хоста и домена, IP-адресов шлюза и DNS-серверов

Имя хоста, которое здесь введено (на рисунке - **myname**), будет использоваться не только протоколами **TCP/IP**, но и собственным протоколом **QNX - Qnet**. Никаких других настроек для **Qnet** не требуется.

Если требуется задать дополнительные IP-адреса интерфейсов и шлюзов, то щелкните на гиперссылке **Click here to toggle the i display of advanced options such as network routing**.

2.3 Настройка локализации

Есть три объекта локализации (русификации) в **QNX**:

- графическая оболочка **Photon**;
- графический псевдотерминал **pterm**;
- консоль.

Так как в **Photon** используется кодировка **Unicode**, в которой есть кириллические шрифты. Нужно настроить только раскладку клавиатуры. Для этого в **Shelf** (см. рис. 2) в разделе **Configure** нажмите кнопку **Localization**. Запустится программа **User's Configuration**. В ее окне можно выбрать соответствующую вкладку для установки часового пояса, времени и даты. Вкладку **Language** можно не трогать - русского языка там нет. Зато есть вкладка **Keyboard**. Откройте эту вкладку и выберите строку **Russian**. Не забудьте нажать кнопки **Apply** (применить изменения) и **Done** (закрыть окно).

Программа **User's Configuration** считает, что пользователь использует либо английскую, либо русскую раскладку. А если все же нужна и английская раскладка, надо выполнить следующие действия:

открыть файл `/etc/system/trap/.KEYBOARD.myname` (обратите внимание: **myname** - это имя нашей машины);

отредактировать его так, чтобы он содержал две строки:

`en_US_101.kbd`

`ru_RU_102.kbd`

Раскладка, указанная первой, будет использоваться по умолчанию. Обратите внимание, что имя файла начинается с точки (.). Это означает, что файл скрытый, и средства просмотра файловой системы по умолчанию его не отображают. Чтобы видеть скрытые файлы в штатном файловом менеджере графической оболочки **Photon**, выберите элемент **Preferences** меню **Edit** и в открывшемся окне снимите самый верхний флажок **Hide 'dot' Files** (скрывать файлы, имя которых начинается с точки) (рис. 8).

Теперь при запуске приложений графической оболочки **Photon** можно будет вводить как русский, так и английский текст (для переключения между раскладками клавиатуры используется комбинация клавиш **<Alt>+<Shift>** (слева)).

Однако в командной строке (как консоли, так и псевдотерминала) нельзя будет ни вводить, ни читать русские буквы. Для решения этой проблемы предназначен пакет **SWD Cyrillic Pack**, поставляемый с пошаговой инструкцией на русском языке.

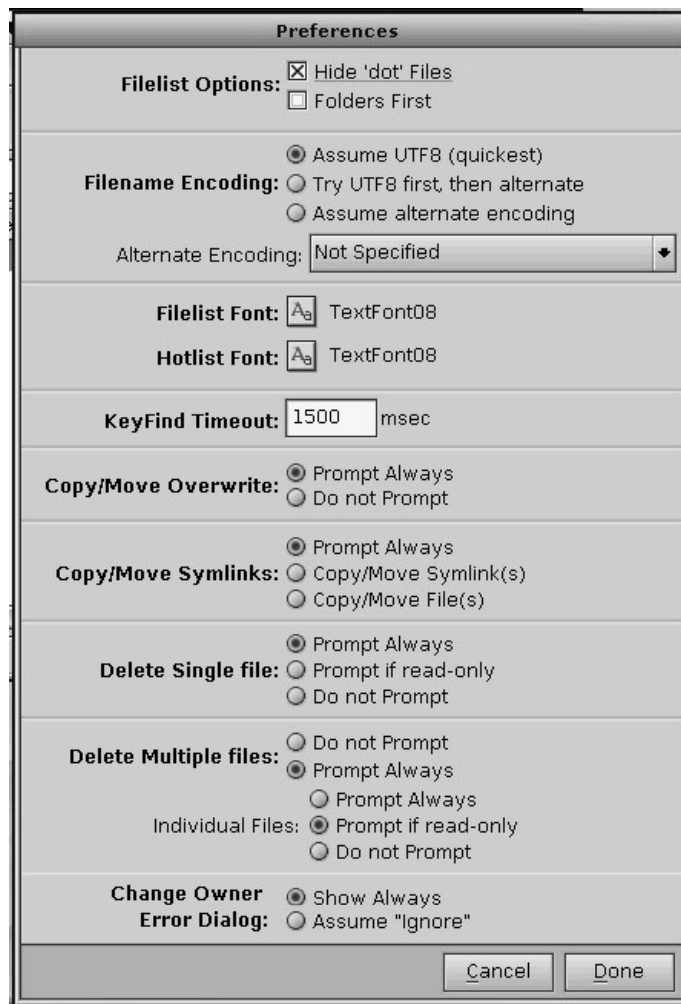


Рис. 8. Окно Preferences файлового менеджера pfm

2.4 Дополнительные настройки

Photon, как и всякая более-менее функциональная графическая оболочка, имеет различные средства конфигурирования, которые нетрудно отыскать, нажав кнопку **Launch** и выбрав меню **Configure**. (Разберитесь самостоятельно с этими утилитами).

Если требуется, чтобы какие-то команды выполнялись при каждом старте **QNX Neutrino**, то создается файл `/etc/rc.d/rc.local`, доступный по исполнению для администратора (пользователя `root`):

```
touch /etc/rc.d/rc.local
chmod 744 /etc/rc.d/rc.local
```

Затем этот файл открывается в любом текстовом редакторе и в него вписываются в виде сценария те команды, которые должны выполняться при каждом запуске **QNX Neutrino**.

Если требуется, чтобы какие-то приложения автоматически запускались при старте **Photon**, то создается файл `$HOME/.ph/phapps` с атрибутом "исполняемый" для владельца:

```
touch ~/.ph/phapps
chmod 744 ~/.ph/phapps
```

Затем этот файл открывается в любом текстовом редакторе и в него вписываются в виде сценария те команды, которые должны выполняться при каждом запуске оболочки **Photon** от вашего имени. Например, программа чтения электронной почты.

Примечание. Сценарий `phapps`, в отличие от сценария `rc.local`, пишется для каждого пользователя отдельно.

Обычно файловый менеджер запускает программу-обработчик при двойном щелчке на имени файла. Если обработчик по умолчанию не подходит, то на имени файла можно щелкнуть правой кнопкой мыши и в открывшемся меню выбрать подходящий элемент (тоже "обработчик", но не по умолчанию). Файловый менеджер **pfm** позволяет изменять

обработчики, соответствующие элементам меню **Open**, **View** и **Edit**. Обработчик **Open** является обработчиком по умолчанию, т. е. вызывается двойным щелчком на имени файла. Для настройки обработчиков надо выбрать элемент **Associations** меню **Edit** - откроется соответствующее окно (рис. 9).

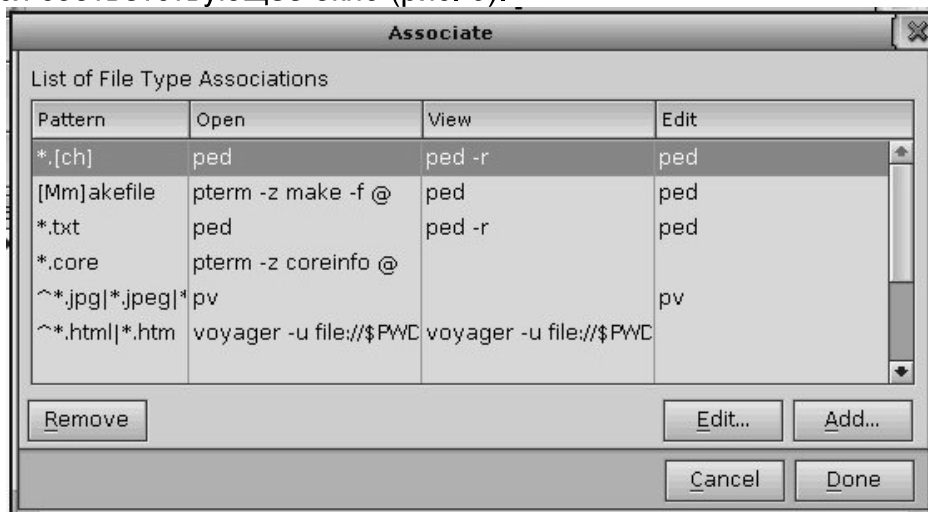


Рис. 9. Привязка программ-обработчиков к именам файлов

Окно ассоциаций представляет собой таблицу, в которой для файлов с различными расширениями имени задаются программы-обработчики. Для добавления новых ассоциаций нажмем кнопку **Add**. В открывшемся окне (рис. 10) заполним соответствующие поля. Например:

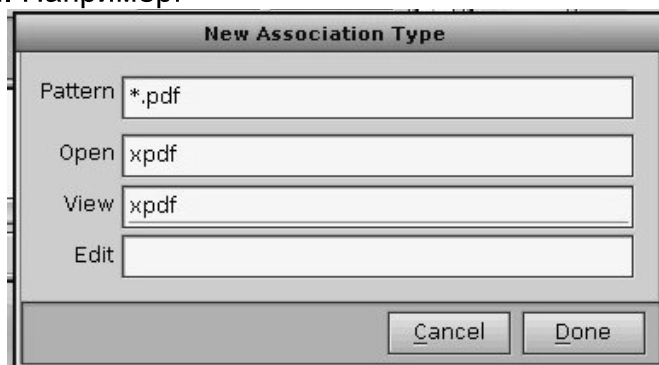


Рис. 10. Ассоциируем PDF-файлы с программой xpdf

Если нужно установить одинаковые обработчики для файлов с разными расширениями имени, то в поле **Pattern** (шаблон) можно перечислить шаблоны имен через логическое "И" (знак конъюнкции - "|"). Не забывайте нажимать кнопку **Done**.

Примечание. Программа **xpdf** входит в пакет **XPDF**, доступный в репозитории **3rd-Party Software**. Для ее работы необходимо из того же репозитория скачать пакет **XPhoton** и запустить сервер **XPhoton** на своей ЭВМ. **XPhoton** удобно запускать из сценария **phapps**.

3 Интерфейс командной строки

3.1 Командные интерпретаторы QNX

С командным интерпретатором можно работать как в консоли, так и в графической среде. Для использования командного интерпретатора в графической оболочке **Photon** нужно запустить псевдотерминал **pterm**. Командный интерпретатор выведет на экран приглашение (для "простого" пользователя - \$, для суперпользователя - #). В ответ на это приглашение можно напечатать команду и нажать клавишу <Enter>. Интерпретатор выполнит команду (либо сообщит об ошибке, если ему это не удалось) и вновь выведет приглашение для ввода следующей команды. И так до тех пор, пока не получит команду **exit** (выход).

Есть несколько командных интерпретаторов:

Bourne Shell (sh) - интерпретатор, стандартизованный в рамках **POSIX 1003.2**;

Korn Shell (ksh) - расширение **sh**;

Bourne Again Shell (bash) - модернизированный интерпретатор **sh**;

C-Shell (csh) - интерпретатор с Си-подобным синтаксисом сценариев и др.

В ОС **QNX** интерпретатором по умолчанию является **ksh**. Поскольку **ksh** полностью реализует функциональные возможности **sh**, ради экономии места, с одной стороны, и по традиции - с другой, в **QNX** файл **/bin/sh** - символическая ссылка на **/bin/ksh**. В одной системе можно одновременно использовать любые интерпретаторы.

Бывает, что использовать интерпретатор **ksh** в системах с ограниченными ресурсами - "дорогое" удовольствие. Поэтому в составе **QNX** есть и "урезанные" интерпретаторы:

- **esh** - встраиваемый командный интерпретатор (**Embedded Shell**). Предоставляет подмножество функциональных возможностей **ksh**, при запуске выполняет простые команды из файла **/etc/esh**. Этот интерпретатор не позволяет писать программы с условными операторами и т. п.;

- **uesh** - малый встраиваемый интерпретатор (**Micro-embedded Shell**), получен в результате урезания **esh**;

- **fesh** - "жирный" встраиваемый интерпретатор (**Fat embedded shell**). В противоположность **uesh**, **fesh** является расширением **esh**. Этот вариант **shell** появился относительно недавно из-за того, что аппаратура современных встраиваемых систем достаточно производительна, чтобы не использовать столь "слабые" интерпретаторы, как **esh** и **uesh**.

3.2 Соглашения по работе с командной строкой

В **QNX** приняты традиционные для **UNIX**-систем соглашения об интерпретации аргументов командной строки. Аргументы подразделяются на две категории: *опции* (или *флаги*) и все остальные. Опции меняют поведение программы, а остальные аргументы содержат разного рода входные данные (например, названия файлов). Опции бывают двух видов:

- *короткие опции* состоят из дефиса и одиночного символа (обычно это буква в нижнем или верхнем регистре). Такие опции быстрее и проще набирать;

- *длинные опции* состоят из двух дефисов, после которых следует имя, содержащее буквы нижнего и верхнего регистров и дефисы. Такие опции легче запоминать и читать (например, в командных сценариях).

Если какая-либо утилита содержит подсказку о своей работе, то эту подсказку можно получить при помощи команды

```
use
```

Например:

```
# use mount
```

В результате выполнения этой команды на экран выдается информация о порядке использования утилиты **mount**. В остальных случаях необходимо обращаться к справочной системе **QNX**.

3.3 Специальные символы командной строки

При запуске с "обычной" программой связывается три так называемых стандартных потока ввода/вывода. Каждый из этих потоков представлен в виде файлового дескриптора:

0 - стандартный ввод (**stdin**);

1 - стандартный вывод (**stdout**);

2 - стандартный поток диагностических сообщений (**stderr**), обычно его называют "стандартным потоком ошибок".

Эти потоки нужны для того, чтобы могли работать стандартные функции языка **C** и **C++**, которые считывают информацию с консоли и выводят что-либо на экран. Поток 0 по умолчанию связан с клавиатурой, а потоки 1 и 2 - с экраном терминала.

Для перенаправления потоков используются следующие специальные символы:

> **файл** Стандартный вывод перенаправляется в **файл**;

>> **файл** Данные из стандартного вывода добавляются в **файл**;

< **файл** Стандартный ввод перенаправляется в **файл**;

p1 | **p2** Вывод команды **p1** направляется на вход команды **p2**;

n > **файл** Перенаправление вывода из файла с идентификатором **n** в **файл**;

n >> **файл** Тоже, что и в предыдущей строке, но данные добавляются к содержимому **файла**;

n > **&m** Объединение потоков с идентификаторами **n** и **m**;

* ? [] Метасимволы, используются для создания шаблонов;

& Выполнение команды в фоновом режиме

; Точка с запятой разделяет команды в командной строке

\ Отменяет специальное значение символов, таких как * ? [] & ; < > |;

' ' Отменяют значение пробела как разделителя и специальное значение всех символов;

" " Отменяют значение пробела как разделителя и специальное значение всех символов за исключением \$ и \;

` ` Позволяет использовать вывод команды как аргументы для другой;

\$ Используется с позиционными параметрами и определенными пользователем переменными.

3.4. Встроенные переменные

CDPATH Путь поиска для команды **cd**;

HOME Аргумент по умолчанию (входной каталог) для команды **cd**;

PATH Путь поиска команд (список директорий, где интерпретатор будет искать введенную вами команду);

MAIL Если эта переменная определяет имя почтового файла, то интерпретатор **shell** информирует пользователя о прибытии почты в указанном файле;

TMOUT Время бездействия юзера в системе, через которое он автоматически будет отключен;

PS1 Первичный вид интерпретатора командой строки по умолчанию "\$";

PS2 Вторичный вид интерпретатора командой по умолчанию ">";

SHELL Имя используемого интерпретатора.

3.5. Основные команды консоли

Команда

pwd [-w]

выводит на экран имя рабочей директории.

```
# pwd
/root
```

Команда

hostname

выводит на экран имя локальной машины (машины, на которой вы работаете).

```
# hostname
localhost
```

Команда

id [имя_пользователя]

выводит пользовательский идентификатор (uid) и идентификатор группы (gid), а так же - эффективный идентификатор (если он отличается от реального) и дополнительные группы.

```
# id root
uid=0(root) gid=0(root) groups=0(root),1(bin),3(sys),4(adm),5(tty)
```

Команда

date

выводит на экран текущее время операционной системы, время и часовой пояс.

```
# date
Wed Jan 08 17:51:47 MSK 2014
```

Команда

time

определяет количество времени, которое занимает выполнение другого процесса. Например, можно замерить время вывода списка файлов текущей директории:

```
# time ls
.      .lastlogin  .ph
..     .mozilla   .profile
0.04s real    0.00s user    0.00s system
```

или замерить время выполнения команды, вызывающей десятисекундную задержку

```
# time sleep 10
10.04s real    0.00s user    0.00s system
```

Команда

who

выводит список пользователей, зарегистрированных в системе (кроме текущего).

who am I

выводит имя текущего пользователя и имена его терминала

```
# who am i
root          ttyt0      8 Jan 17:58
```

Команда

history

выводит последние (1000 или более) команд, выполненных в командной строке текущим пользователем

```
# history
1      use mount
2      pwd
3      hostname
4      id root
5      date
6      time ls
7      time sleep 10
8      who
9      who am i
```

Команда

uptime

выводит количество времени, прошедшее с последней перезагрузки

```
# uptime
17:59:38 up 16 min, 0 users
```

Команда

ps

выводит список всех процессов, исполняемых в данный момент

```
# ps
  PID TTY          TIME CMD
 426003 ?            00:00:00 pterm
 180246 ?            00:00:00 login
 180247 ?            00:00:00 login
 180248 ?            00:00:00 login
 258073 ?            00:00:00 login
 331802 ?            00:00:00 pwm
 426013 ?            00:00:00 sh
 360478 ?            00:00:00 shelf
 380960 ?            00:00:00 bkgdmgr
 380961 ?            00:00:00 wmswitch
 380962 ?            00:00:00 saver
 499747 ?            00:00:00 ps
```

Команда

top

выводит список процессов, выполняемых в системе в данный момент, отсортированный по использованию процессора.

PID = идентификатор процесса (process identification).

USER=имя пользователя - владельца процесса (запустившего процесс).

PRI=приоритет (priority) процесса (чем больше число, тем ниже приоритет, нормально - 0, высший приоритет=-20, низший=20).

NI=уровень любезности (niceness level) (т.е., если процесс пытается быть любезным, уменьшая свой приоритет на указанное число). Чем больше число, тем больше любезность процесса (т.е., приоритет ниже).

SIZE=килобайты кода+данных+стека занимаемых программой.

RSS=килобайты физической (на микросхеме) памяти, занимаемой процессом.

SHARE=килобайты памяти, разделяемой с другими процессами.

STAT=состояние процесса: S – спит (sleeping), R – выполняется (running), T – приостановлен (stopped) или трассируется (traced), D – беспрепятственно спит (uninterruptable sleep), Z – зомби, ни жив, ни мертв (zombie).

%CPU=процент использования процессора (с момента последнего обновления экрана).

%MEM=процент использования физической памяти.

TIME=общее время использования процессора (с момента старта).

COMMAND=командная строка, запустившая процесс (поосторожнее с паролями, и т.п., в командной строке, потому что все, кто наберет "top", смогут это увидеть!

Нажмите <Ctrl>+<C>, после просмотра.

```
35 processes; 95 threads;
CPU states: 99.3% idle, 0.4% user, 0.1% kernel
Memory: 0 total, 1G avail, page size 4K

  PID   TID PRI  STATE   HH:MM:SS   CPU  COMMAND
  229404 3 12 Rply    0:00:02   0.39% io-graphics
    1    6 10 Run     0:00:00   0.11% kernel
  151567 1 10 Rcv     0:00:00   0.05% devc-pty
  507939 1 10 Rply    0:00:00   0.01% top
  426003 1 10 Rcv     0:00:00   0.01% pterm
  360478 2 10 CdU    0:00:00   0.00% shelf
  360478 1 10 Rcv     0:00:00   0.00% shelf
   4101 5 10 NSlp    0:00:00   0.00% io-usb
   8200 7 10 Rcv     0:00:00   0.00% devb-eide
   8200 3 21 Rcv     0:00:00   0.00% devb-eide

CPU idle:      Min      Max      Average
Mem Avail:    1704MB    1704MB    1704MB
Processes:      35      35      35
Threads:       95      95      95
```

Команда

uname

выводит имя операционной системы

```
# uname
QNX
```

Команда

df

выводит свободное место на всех смонтированных файловых системах

```
# df
/dev/hd0t179      8385832    2507356    5878476      30% /
/dev/hd0          8388608    8388608         0     100%
```

Команда

du

отображает использование дискового пространства всеми поддиректории, начиная с "/" (корневой) директории

```
# du
4 ./ph/wm
8 ./ph
7 ./mozilla/firefox/3dx0n6pq.default/chrome
2 ./mozilla/firefox/3dx0n6pq.default/extensions
43 ./mozilla/firefox/3dx0n6pq.default/Cache
16 ./mozilla/firefox/3dx0n6pq.default/bookmarkbackups
5583 ./mozilla/firefox/3dx0n6pq.default
5586 ./mozilla/firefox
5588 ./mozilla
5599 .
```

Команда

set

выводит значения переменных окружения пользователя (всех). Обычно вывод занимает более одного экрана.

Команда

echo [-n] [аргумент]

выводит аргументы командной строки на стандартный вывод, разделяя их пробелами и завершая вывод переводом строки. Флаг -n предоставляет возможность отменить перевод строки после вывода аргументов

```
# echo -n Hello!  
Hello!# _
```

Команда

sysctl

выводит или устанавливает все конфигурируемые параметры ядра QNX

```
# sysctl  
usage: sysctl [-dne] [-x[x!]-r] variable ...  
sysctl [-ne] [-q] -w variable=value ...  
sysctl [-dne] -a  
sysctl [-dne] -A  
sysctl [-ne] -M  
sysctl [-dne] [-q] -f file
```

Команда

kill

завершает процесс (команда POSIX).

Команда

slay

завершает процесс (команда QNX).

Команда

sleep время

задерживает выполнение команды на время, заданное в секундах.

Команда

nice команда

понижение приоритета команды. Позволяет выполнить другую команду, с более низким приоритетом.

Команда

tty

получение имени текущего терминала

```
# tty  
/dev/ttyp0
```

Команда

stty

установка функций терминала. Позволяет узнать состояние любого терминала и настроить его на требуемый режим работы

```
# stty  
Name: /dev/ttyp0  
Type: pseudo  
Opens: 3  
+edit +echok  
+osflow  
intr=^C quit=^\ erase=^? kill=^U eof=^D start=^Q stop=^S susp=^Z  
lnext=^U min=01 time=00 pr1=^I pr2=5B left=44 right=43 up=41  
down=42 ins=40 del=50 home=48 end=59
```

Команда

cron

демон для запуска команд по расписанию.

Команда

exit

выход из терминала (консоли).

Команда

passwd

изменяет пароль пользователя.

Команда

su

получение прав другого пользователя.

Команда

wait

используется, когда нужно подождать завершения какого-то процесса. Появление

приглашения после ввода команды wait указывает на завершение всех запущенных ранее процессов.

Команда

pidin

информация о процессах и потоках.

Команда

fg

переводит процесс из фона на передний план. Если эта команда была вызваны без указания номера процесса, то она воздействуют на текущий исполняющийся процесс.

Команда

bg

перезапускает приостановленный процесс в фоновом режиме. Если эта команда была вызваны без указания номера процесса, то она воздействуют на текущий исполняющийся процесс.

Команда

tee

дублирует стандартный вывод. Команда читает информацию из стандартного ввода и выводит ее одновременно на терминал и в заданные файлы.

Команда

cd

смены текущего каталога

```
# pwd
/root
# cd /tmp
# pwd
/tmp
```

Команда

cp

копирования файлов и каталогов

```
# cp myfile myfile2
# ls
.      ..      myfile  myfile2
```

Команда

head

вывода первых строк файла. По умолчанию первые 10 строк.

```
# head myfile
Hello!
```

Команда

mv

переименования (перемещения) файлов или каталогов

Команда

rm

удаления файлов

Команда

rmdir

удаления каталогов (только пустых)

Команда

tail

вывода концевой части файла. По умолчанию последние 10 строк.

```
# tail myfile
Hello!
Hi
```

Команда

ls

вывода на экран содержимого каталога.

Команда

mkdir

создания нового каталога

```
# mkdir myfolder
# ls
.  ..  myfile  myfolder
```

Команда
cat

слияния (конкатенации) файлов (если указано одно имя команда выводит содержимое файла в терминал)

```
# cat myfile
Hello!
Hi
```

Команда
pr

печати содержимого одного или нескольких файлов с автоматическим разбиением на страницы и с заголовком на каждой странице.

Команда
lpr

печати файлов одновременно с выполнением других команд.

Команда
cmp

сравнения файлов и выдача отчета о различиях

```
# cmp myfile myfile2
myfile myfile2 differ: char 2, line 1
```

Команда
find

поиска файлов по заданным условиям. Просматриваются рекурсивно все подкаталоги для каждого указанного каталога и ищутся файлы, отвечающие заданным условиям.

Команда
dcheck

проверка корректности файловой системы. Проверяется правильность списков свободных и используемых блоков, выводится общее число файлов, каталогов, число используемых блоков, число свободных блоков и т.д.

Команда
mount

монтирования файловой системы.

Команда
umount

демонтирования файловой системы.

Команда
comm

поиска одинаковых и разных строк в файлах.

Команда
grep

поиска строк с заданным шаблоном. Служит для поиска соответствующих выражений (строк) в одном или нескольких файлах.

Команда
file

определение типа файла (.OBJ, .C, ASCII и т.д.).

Команда
split

разбиение файла на части по n строк (по умолчанию n=1000). Если задано имя выходного файла, то генерируется последовательность файлов с данным именем и буквами aa, ab, ac,... в конце. Если имя выходного файла не задано, используется имя "x".

Команда
wc

подсчет числа строк, слов и символов в одном или более файлов. Строки в файле разделяются символом "\n", слова - пробелами, горизонтальной табуляцией или переводом строки.

Команда
uniq

вывод одинаковых строк файла. Находит одинаковые соседние строки файла. По умолчанию все одинаковые строки кроме одной удаляются.

Команда
diff

обнаружения различий в файлах. Определяются изменения, которые должны быть произведены в файлах, чтобы сделать их идентичными. Позволяет экономить место при хранении ряда версий файла.

Команда
sort

сортировка и слияние файлов. Соединяет и сортирует файлы, помещает результат в заданный файл. Если ключ сортировки не задан, при сравнении используется вся строка.

Команда
ln

создает ссылку на файл. Ссылка выглядит как копия исходного файла, но в действительности существует только один экземпляр файла, но с двумя (или более) входными точками. Соответственно, любые изменения в файле видны сразу с двух точек. Когда одна из ссылок удаляется, другая (другие) останется (останутся) нетронутыми.

3.6. Примеры использования

Файловая система операционных систем семейства **QNX** имеет общепринятую для **UNIX**-подобных систем древовидную структуру. Корнем этого дерева является каталог, обозначаемый символом / (слэш) и называемый "**root**".

Теперь перейдем в другой каталог, например, в **/tmp**:

cd /tmp

Обратите внимание, что указано *абсолютное* *путевое имя* каталога, в который осуществляется переход. Путевое имя - это полное имя каталога или файла, однозначно указывающее место каталога (файла) в дереве файловой системы начиная от каталога "**root**". Имя можно указывать и *относительное*, т. е. относительно *текущего* каталога. Если имя начинается не с /, то **Neutrino** считает, что указано относительное имя. Это значит, что если бы мы в предыдущем примере ввели такую команду:

cd tmp

то операционная система решила бы, что мы хотим перейти в каталог **/root/tmp**. Чтобы из любого места файловой системы вернуться в домашний каталог, надо ввести команду **cd** без аргументов. Чтобы перейти в вышестоящий каталог (т. е. приблизиться к каталогу /), надо ввести такую команду:

cd ..

Можно создавать свои каталоги, например, создадим каталог **/tmp/myfolder**. Если мы находимся в каталоге **/tmp**, то достаточно указать относительный путь:

mkdir myfolder

Примечание. При запуске программы интерпретатор перебирает по очереди все каталоги, перечисленные через двоеточия в переменной окружения **path**. То есть запускаемая программа должна находиться в одном из каталогов, указанных в переменной **PATH**, в противном случае программа просто не будет найдена. Значение этой переменной можно посмотреть с помощью команды:

echo \$PATH

Значения всех переменных окружения можно посмотреть с помощью команды **set** без аргументов.

Теперь создадим в новом каталоге файл **myfile**. Из каталога **/tmp** это можно сделать так:

touch myfolder/myfile

или выполнить две команды:

cd myfolder

touch myfile

или указать абсолютное путь к создаваемому файлу:

```
touch /tmp/myfolder/myfile
```

Посмотрим на содержимое каталога `/usr/bin`:

```
ls /usr/bin
```

Создадим копию файла `/etc/services`, присвоив ей имя `/tmp/myfolder/myfile_2`:

```
cp /etc/services /tmp/myfolder/myfile_2
```

Переименуем `/tmp/myfolder/myfile_2` в `/tmp/myfolder/myfile_3`:

```
mv /tmp/myfolder/myfile_2 /tmp/myfolder/myfile_3
```

Посмотрим содержимое файла `/tmp/myfolder/myfile_3`:

```
cat /tmp/myfolder/myfile_3
```

Если содержимое файла не уместилось на экране, можно воспользоваться "конвейером":

```
cat /tmp/myfolder/myfile_3 | more
```

Теперь можно ограничивать вывод на экран фрагментами, по размеру соответствующими доступному пространству экрана, нажимая клавишу `<пробел>` (прекратить вывод - клавиша `<Q>`).

Удалим файл `/tmp/myfolder/myfile`:

```
rm /tmp/myfolder/myfile
```

Удалим каталог `/tmp/myfolder`:

```
rmdir /tmp/myfolder
```

Команда `rmdir` умеет удалять только пустые каталоги. Уничтожить непустой каталог поможет следующая команда:

```
rm -R /tmp/myfolder
```

Можно задать несколько команд в одной командной строке, разделив их точкой с запятой (;). Если команда длинная, и ее необходимо перенести на следующую строку, то перед переводом строки ставят обратный слеш (\). Чтобы запустить команду в фоновом (неинтерактивном) режиме, в ее конце ставится амперсанд (&). Например, запустим утилиту `ped` - штатный текстовый редактор **Photon** из окна псевдотерминала:

```
ped &
```

Интерпретатор сообщит номер фонового процесса и выдаст, приглашение ввести новую команду не дожидаясь завершения фоновой программы.

Если вы забыли добавить амперсанд, введя только

```
ped
```

то можно остановить интерактивный процесс, нажав комбинацию клавиш `<Ctrl>+<Z>`, а затем перевести его в фоновый режим с помощью команды `bg`. Перевод последней программы, запущенной в фоновом режиме, в интерактивный режим выполняется с помощью команды `fg`.

Если требуется, чтобы фоновый процесс завершился прежде, чем будут выполнены какие-либо действия, используется команда `wait`. Если в качестве аргумента для `wait` указать **PID**² конкретного процесса, то она будет ждать именно его завершения, а если этот параметр не задавать, то `wait` будет ожидать завершения всех фоновых процессов, дочерних для данного **shell**.

Интерактивный процесс можно принудительно завершить, используя комбинацию клавиш `<Ctrl>+<C>`, а фоновый процесс - с помощью **POSIX**-утилиты `kill` или **QNX**-утилиты `slay`. В качестве аргумента для утилиты `kill` требуется указать **PID** уничтожаемого процесса, а для утилиты `slay` - имя уничтожаемого процесса. Идентификатор процесса, в свою очередь, можно получить либо с помощью **POSIX**-утилиты `ps`, либо с помощью **QNX**-утилиты `pidin`.

Двойной амперсанд `&&` предписывает интерпретатору **shell** выполнять следующую команду при условии нормального завершения предыдущей, иначе - игнорировать. Например:

² **Process ID** - идентификатор процесса.

```
mkdir /tmp/myfolder2 && touch /tmp/myfolder2/myfile2
```

В этом случае сначала будет предпринята попытка создать каталог /tmp/myfolder2, а затем (если удалось создать каталог) в нем будет создан файл myfile2.

Символ ||, напротив, предписывает интерпретатору **shell** выполнять следующую команду при ненормальном завершении предыдущей, иначе - игнорировать. Например:

```
mkdir /tmp/myfolder2 || rm /tmp/*
```

В этом случае сначала будет предпринята попытка создать каталог /tmp/myfolder2, и, если каталог создать не удалось, - будет удалено содержимое каталога временных файлов /tmp.

Потоки ввода/вывода и конвейеры

Командный интерпретатор позволяет перенаправлять стандартные потоки ввода, т. е. связывать их с какими-либо файлами. Например:

```
ls /usr > /tmp/file1
```

Команда **ls** сформирует список содержимого каталога /usr и вместо того, чтобы вывести результат на экран, поместит его в файл /tmp/file1. Если файла с таким именем не было, то он будет создан. Если же файл /tmp/file1 уже существовал, то его прежнее содержимое будет уничтожено.

Выполним другую команду:

```
pwd > /tmp/file1
```

Команда **pwd** сформирует полное имя текущего каталога и поместит его в файл /tmp/file1, уничтожив прежнее содержимое файла. Если нужно добавить новую информацию к прежней, используется директива >>, т. е. предыдущая команда будет выглядеть так:

```
pwd >> /tmp/file1
```

В этом случае вывод утилиты **pwd** будет добавлен в конец содержимого файла /tmp/file1.

Для перенаправления стандартного ввода используется символ <. Например:

```
wc -c < /tmp/file1
```

Команда **wc** подсчитает и выдаст на экран число символов в файле /tmp/file1.

При перенаправлении следует указывать номер перенаправляемого потока, однако **shell** по умолчанию знает, что ">" - это "1>", а "<" - это "0<". Если же требуется оперировать другими потоками (например, с **stderr**), то значение дескриптора нужно задавать явно. Проиллюстрируем:

```
ls /tmp11
```

Если каталог /tmp11 не существует, то команда выдаст сообщение об ошибке:

```
ls: No such file or directory (/tmp11)
```

Допустим, мы хотим все диагностические сообщения команды "сбрасывать" в файл /tmp/errlog. Тогда команда должна выглядеть так:

```
ls /tmp11 2 > /tmp/errlog
```

Если же сообщения об ошибках не интересуют, чтобы не "засорять" ими экран, то сделаем так:

```
ls /tmp11 2 > /dev/null
```

Потоки можно объединять, используя конструкцию >&. Для иллюстрации выполним команду, пишущую в стандартные потоки вывода и ошибок. Для этого попытаемся вывести на экран содержимое существующего файла (/tmp/file1) и несуществующего файла (/tmp/file2), а стандартный вывод направим в файл /tmp/log:

```
cat /tmp/file1 /tmp/file2 > /tmp/log
```

Теперь модифицируем команду так, чтобы поток ошибок направлялся туда же, куда и поток вывода:

```
cat /tmp/file1 /tmp/file2 > /tmp/log 2>&1
```

В приведенной команде поток 2 добавляется к потоку 1.

Допускается произвольно комбинировать операции перенаправления потоков в одной команде:

```
wc -c </tmp/file1 >/tmp/file2
```

В данном случае утилита `wc` осуществит ввод из файла `/tmp/file1`, а результат запишет в файл `/tmp/file2`.

Содержимое файла `/tmp/file1` можно обнулить:

```
cat >/tmp/file1 < /dev/null
```

Вывод одной программы можно объединять с вводом другой, с помощью символа `|` (конвейер):

```
ls -l /usr/bin | wc -l
```

В данном примере утилита `ls` сформирует вывод, состоящий из информации о содержимом каталога `/usr/bin`, результат будет передан на "вход" утилиты `wc`, которая отобразит на экране количество строк в выводе `ls`.

Если программа "умеет" работать и со стандартным вводом, и со стандартным выводом, то ее называют фильтром (типичный фильтр - утилита `wc`). В конвейере можно использовать несколько команд, а значит, все команды конвейера, кроме первой и последней, должны быть фильтрами. Например:

```
ls /usr/bin | sort | wc -l > /tmp/file2
```

Утилита `tee`, позволяет дублировать поток вывода. Один из потоков вывода, например, можно записать в файл, а другой – вывести на экран:

```
ls -l /usr | tee /tmp/lslog
```

Утилита `tee` направляет свой стандартный ввод на свой стандартный вывод, дублируя его в указанные файлы (в примере задан только один файл - `/tmp/lslog`).

Устройства в **QNX** представлены в виде специальных файлов, а значит, механизм перенаправления потоков можно использовать для "сырых" операций чтения/записи с устройствами. Под "сырыми" в данном случае подразумеваются такие операции, при выполнении которых пользователь сам отвечает за то, что формат передаваемых им данных будет понятен драйверу, а формат вывода драйвера будет понятен ему самому.

Например, выведем сообщение `"Hello folks!"` в окно псевдотерминала `pterm`. Для этого в одном окне `pterm` введем команду `tty`. В ответ мы получим имя файла псевдотерминала, ассоциированного с данной копией `pterm`. Пусть это будет `/dev/ttyp1`. Тогда в любом другом окне `pterm` введем команду:

```
echo Hello folks! > /dev/ttyp1
```

Можно передавать результат выполнения одной команды в качестве аргумента другой команде, для этого команду-"аргумент" следует заключить в одиночные косые кавычки (```).

Примечание. Клавиша этого символа обычно находится под клавишей `<Esc>` в левой верхней части клавиатуры - не путайте с клавишей апострофа (`'`), расположенной рядом с клавишей `<Enter>`.

Проиллюстрируем:

```
less 'which ph'
```

Команда `which` ищет указанный файл по очереди во всех каталогах, перечисленных в переменной окружения `PATH` (т. е. там же, где ищет команду интерпретатор **shell**), как только найдет - выводит на экран полное имя найденного файла и прекращает поиск. Команда `less` позволяет "прокручивать" длинные текстовые файлы, дозируя их фрагментами, равными доступному пространству экрана (прокручивание выполняется нажатием клавиши `<пробел>`, выход - `<Q>`). То есть указанная команда позволяет просмотреть текст сценария `ph`, не задумываясь о том, где он физически находится.

Таким образом, перенаправление, слияние, объединение потоков ввода/вывода предоставляет администратору мощный инструмент управления системой. Этот механизм подобен технологиям компонентного программирования, получившим широкое распространение в последние годы. При этом компонентами являются любые программы, как входящие в дистрибутив, так и написанные самим пользователем. **Shell** же позволяет сочетать их произвольным образом.

Рассмотрим такой пример: пусть требуется получить список файлов каталога `/bin`, являющихся символическими ссылками, причем для каждого файла в этом списке

должна быть указана контрольная сумма.

Можно написать программу, которая будет делать эту работу, а можно использовать штатные утилиты **QNX**. Поиск файлов по заданным критериям выполняет утилита **find**:

```
find /bin -type 1
```

Контрольное суммирование выполняет утилита **cksum**. Она принимает аргумент - список объектов, поэтому передадим ей в качестве такого аргумента результат выполнения утилиты **find**:

```
cksum 'find /bin -type 1'
```

Форматирование потока вывода выполняет утилита **awk**. От **awk** нам нужна возможность оставить в выводе утилиты **cksum** только первый и третий столбцы, разделенные символом табуляции, причем сначала выведем третий столбец (имя файла), а затем - первый (контрольная сумма). Результат сохраним в файле **result.txt**. Итак:

```
cksum 'find /bin -type 1' | awk '{print $3 «\t» $1}' > \
/tmp/result.txt
```

С помощью стандартных **POSIX**-утилит и командного интерпретатора можно легко решать достаточно разнообразные и сложные задачи.

4 Содержание отчета по лабораторной работе

Отчет по лабораторной работе должен содержать следующие сведения:

цель работы;

примеры выполнения команд;

пояснительные комментарии к командам.